Problem Statement #31

Participant Name	Kalakuntla Sunil Gopi
Email ID	kalakuntla.sunilgopi@wipro.com
Problem Title	**Employee Attendance & Leave Management**
Domain	HR

Problem Description

Build a Employee Attendance & Leave Management in Python (HR domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the HR domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #32

Participant Name	Gandra Abhinaya
Email ID	gandra.abhinaya@wipro.com
Problem Title	**Food Delivery Order Tracker**
Domain	Food Tech

Problem Description

Build a Food Delivery Order Tracker in Python (Food Tech domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Food Tech domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #33

Participant Name	Sathyamoorthy J
Email ID	sathyamoorthy.j@wipro.com
Problem Title	**Event Management & Ticketing System**
Domain	Events

Problem Description

Build a Event Management & Ticketing System in Python (Events domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Events domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #34

Participant Name	SIMRITI R
Email ID	simriti.r@wipro.com
Problem Title	**Online Auction & Bidding Platform**
Domain	E-Commerce

Problem Description

Build a Online Auction & Bidding Platform in Python (E-Commerce domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the E-Commerce domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #35

Participant Name	Ankita Basak
Email ID	ankita.basak1@wipro.com
Problem Title	**Fitness Activity & Calorie Tracker**
Domain	Health & Wellness

Problem Description

Build a Fitness Activity & Calorie Tracker in Python (Health & Wellness domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Health & Wellness domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #36

Participant Name	GARLAPATI VENKATA KRISHNA CHAITHANYA
Email ID	krishnachaithanya.garlapati@wipro.com
Problem Title	**Real Estate Property Listing System**
Domain	Real Estate

Problem Description

Build a Real Estate Property Listing System in Python (Real Estate domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Real Estate domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #37

Participant Name	NANDIMANDALAM SAKETH REDDY
Email ID	nandimandalam.reddy@wipro.com
Problem Title	**Compiler Lexical Token Analyzer**
Domain	Computer Science

Problem Description

Build a Compiler Lexical Token Analyzer in Python (Computer Science domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Computer Science domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #38

Participant Name	SURAPANENI POOJITHA
Email ID	poojitha.surapaneni@wipro.com
Problem Title	**Ride Sharing & Cab Booking System**
Domain	MaaS

Problem Description

Build a Ride Sharing & Cab Booking System in Python (MaaS domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the MaaS domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #39

Participant Name	Sabarish B
Email ID	sabarish.b@wipro.com
Problem Title	**Insurance Policy Management System**
Domain	Insurance

Problem Description

Build a Insurance Policy Management System in Python (Insurance domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Insurance domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #40

Participant Name	BALANIVETHA MURUGAN
Email ID	balanivetha.murugan@wipro.com
Problem Title	**Court Case Management System**
Domain	Legal

Problem Description

Build a Court Case Management System in Python (Legal domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Legal domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #41

Participant Name	CHADA VIJAY KANTH REDDY
Email ID	chada.reddy@wipro.com
Problem Title	**Scientific Calculator with History**
Domain	Utilities

Problem Description

Build a Scientific Calculator with History in Python (Utilities domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Utilities domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #42

Participant Name	R DRAVID
Email ID	r.dravid@wipro.com
Problem Title	**Smart Grid Energy Monitoring System**
Domain	Energy

Problem Description

Build a Smart Grid Energy Monitoring System in Python (Energy domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Energy domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #43

Participant Name	Madas Rushikesh
Email ID	rushikesh.madas@wipro.com
Problem Title	**Drone Fleet Management System**
Domain	Aerospace

Problem Description

Build a Drone Fleet Management System in Python (Aerospace domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Aerospace domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #44

Participant Name	ALAHARI VARSHA
Email ID	alahari.varsha@wipro.com
Problem Title	**Cybersecurity Incident Response Tracker**
Domain	Security

Problem Description

Build a Cybersecurity Incident Response Tracker in Python (Security domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Security domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.